
esmc.cpp: A Zero-Dependency, Metal-Accelerated C/C++ Runtime for ESM Cambrian Protein Embeddings

Ananya Pathak¹

Anagh Pathak²

Sakshi Sharma¹

¹<https://ananyapathak.xyz>

²Université de Strasbourg

pathakananya95@gmail.com, a.pathak@unistra.fr, sak5shi.sharma93@gmail.com

Abstract

Protein language models (PLMs) such as ESM Cambrian (ESM-C) produce per-residue embeddings that power variant-effect prediction, function annotation, and structure-adjacent tasks, but their reference implementations require a heavyweight Python and deep-learning stack that is awkward for edge, batch, and embedded deployment. We port ESM-C 300M, an encoder-only PLM, to `esmc.cpp`: a standalone C/C++ inference runtime built directly on the `ggml/llama.cpp` compute substrate, with no Python dependency at inference time. We contribute (i) a GGUF weight schema and converter that maps the native checkpoint layout (fused QKV, fused SwiGLU, biases confined to the pre-QKV LayerNorm, RoPE-NeoX, query/key LayerNorm, residual scaling) to `ggml` graph operations; (ii) a staged, bottom-up validation pipeline (tokenizer $\rightarrow$ layer-0 attention probes $\rightarrow$ full-forward cosine similarity) that is engineered to catch the silent numerical failures characteristic of PLM ports; and (iii) a benchmark study on an Apple M4 Max with 36 GB unified memory. The F16 runtime reproduces PyTorch embeddings to a mean per-residue cosine of 0.99999 across 100 Swiss-Prot sequences, and 8-bit quantization preserves 0.99971 while halving the model file. A completed performance roadmap – flash attention, GPU-resident graph scheduling, and multi-sequence batching – addresses the long-sequence throughput bottleneck and achieves $3.6\times$ single-sequence throughput at batch 16. On Apple Silicon, F16 Metal inference reaches 36.9 seq/s on medium sequences, and all 36 tested model/precision/backend/length configurations fit comfortably within the 36 GB budget. We also document concrete silent-failure case studies, including a single-token vocabulary transposition that corrupted embeddings only for sequences containing asparagine or glutamine.

1 Introduction

Protein language models trained with masked language modeling over hundreds of millions to billions of clustered sequences have become a default featurizer for computational biology. ESM Cambrian (ESM-C) [1] is a recent encoder-only family (300M, 600M, 6B parameters) whose single forward pass yields per-residue embeddings with no autoregressive loop and no growing key/value cache. The official runtime, however, is a PyTorch stack: deploying it on a laptop, an offline batch node, or an embedded system means shipping Python, PyTorch, and a constellation of CUDA/Metal dependencies, which is at odds with reproducibility, cold-start latency, and resource-constrained inference.

Table 1: Architectural deltas between ESM2 and ESM-C that determine port correctness.

Feature	ESM2	ESM-C
FFN	GELU, 2 matrices	SwiGLU, 3 matrices
Biases	present	none (except pre-QKV LayerNorm bias)
Pre-encoder LayerNorm	yes	no
Positional encoding	learned + RoPE	RoPE only (NeoX)
FFN width	$4 d_{\text{model}}$	$\sim 8/3 d_{\text{model}}$ (2560 for 300M)
Context length	1024	2048

The llama.cpp project and its ggml tensor library [6] have demonstrated that decoder-only language models can be run efficiently as a single self-contained binary with on-disk quantization via the GGUF format [7]. We ask whether the same engineering discipline transfers to a *bidirectional*, *encoder-only* protein model, and at what cost to numerical fidelity. This is not a mechanical exercise: ESM-C differs from the widely ported ESM2 family [2] in several details (SwiGLU instead of GELU, biases only on the pre-QKV LayerNorm, RoPE-only positions, a wider context, and a non-standard feed-forward width), and any one of these, if mis-implemented, produces embeddings that are numerically plausible but biologically wrong without raising an error.

This paper makes the following contributions:

- **A systems port.** `esmc.cpp`, a dependency-minimal C/C++ runtime and C library for ESM-C 300M embeddings on the ggml/llama.cpp stack, with CPU and Apple Metal backends and a small command-line tool (`esmc-embed`).
- **A GGUF schema and converter** for ESM-C that ingests the native checkpoint layout, splitting fused QKV and fused SwiGLU projections, and that records the empirically measured feed-forward width rather than a formula (Section 3).
- **A staged validation methodology** that catches silent numerical failures, together with a set of real bug case studies encountered during the port (Section 4).
- **An empirical study** on consumer Apple Silicon spanning numerical correctness, quantization, throughput, memory footprint, a downstream ProteinGym variant-effect task, and a completed performance roadmap incorporating flash attention, GPU-resident graph scheduling, and multi-sequence batching (Section 5).

We deliberately scope the quantitative study to the 300M model on a single M4 Max (36 GB) host; generalization to 600M and 6B is left as future work (Section 6).

2 Background and Related Work

ESM-C versus ESM2. ESM-C is an encoder-only transformer with pre-LayerNorm [5] blocks, rotary position embeddings (RoPE) [3], and a gated SwiGLU [4] feed-forward network. Relative to ESM2 [2], the architectural deltas that govern correctness are summarized in Table 1. The most consequential are the replacement of the GELU two-matrix FFN with a three-matrix SwiGLU FFN, biases being confined to the pre-QKV LayerNorm, RoPE-only positional encoding in the NeoX rotation style, and a feed-forward width of approximately $8/3 d_{\text{model}}$ rather than <

(2560 wide) is k -quant-eligible; the remaining per-layer matrices fall back to legacy 32-element blocks (Section 3).

3 System Design

3.1 Architecture Fidelity

ESM-C 300M has 30 layers, $d_{\text{model}} = 960$, 15 heads (head dimension 64), a SwiGLU intermediate width of 2560, a context length of 2048, a 33-token vocabulary, LayerNorm $\varepsilon = 10^{-5}$, and RoPE $\theta = 10000$. The reference forward pass we reproduce is:

1. token embedding lookup (no positional embedding table; RoPE handles positions);
2. for each layer: pre-LayerNorm; separate Q, K, V projections; per-projection query/key LayerNorm; scaling of Q by $1/\sqrt{d_{\text{head}}}$ *before* RoPE; RoPE-NeoX on Q and K ; bidirectional (non-causal) attention $\text{softmax}(QK^\top)V$; output projection; residual add divided by a residual scale $\sqrt{30/36} \approx 0.9129$; then pre-LayerNorm, SwiGLU FFN $W_{\text{down}}(\text{SiLU}(W_{\text{gate}}x) \odot W_{\text{up}}x)$, and a second scaled residual;
3. a single final LayerNorm producing the per-residue embedding $[n_{\text{embd}} \times n_{\text{tokens}}]$.

There is no pre-encoder or wrap-around LayerNorm, and no token dropout, both of which were present in ESM2.

3.2 GGUF Conversion

The native 300M checkpoint stores fused projections under `esmc.*` keys rather than the HuggingFace `model.layers.*` layout. Our converter (i) splits the fused QKV weight into `attn_q/k/v`, (ii) splits the fused SwiGLU `fc1_weight` into `ffn_gate` and `ffn_up`, (iii) preserves the pre-QKV LayerNorm weight *and bias* and the per-projection query/key LayerNorm weights, and (iv) reads the feed-forward width directly from the tensor shape and writes it to the GGUF metadata key `esmc.feed_forward_length` (2560) instead of computing it from a formula. LayerNorm weights are stored in F32 regardless of the quantization target. The 300M model maps to 363 tensors (3 global plus 12 per layer $\times$ 30: 4 attention projections, 2 attention-norm (weight + bias), 2 query/key-norm, 1 FFN-norm, 3 FFN projections). The full architecture is recorded as GGUF metadata (`esmc.block_count`, `esmc.embedding_length`, `esmc.attention.head_count`, `esmc.rope.freq_base`, etc.) so that the C++ loader is data-driven.

3.

Table 2: Silent-failure bugs encountered during the port. Each produced wrong embeddings without an error.

Bug	Root cause	Effect (after fix)
BUG-001	.npy header omitted trailing newline byte	NaN output $\rightarrow$ valid floats
BUG-002	Q/N tokens swapped (indices 16/17)	medium cosine 0.953 $\rightarrow$ 0.99999
BUG-003	Q scaled at softmax instead of before RoPE	Q norm error 700% $\rightarrow$ 0.0025%
BUG-004	nan <= 0.999 is False in Python (false PASS)	added explicit finiteness checks

Table 3: Numerical fidelity and size by precision (300M, 100 Swiss-Prot sequences, CPU). Pass = per-sequence mean cosine > 0.999 (F16/Q8_0) or > 0.995 (Q4_K_*). File size relative to F16 (634 MiB) in parentheses.

Precision	Aggregate mean cosine	Pass rate	Worst min cosine	Max mean-pool L2	File size
F16	0.999985	100/100	0.999711	0.0030	634 MiB (1.00)
Q8_0	0.999714	100/100	0.994269	0.0164	337 MiB (0.53)
Q4_K_M	0.995966	91/100	0.940125	0.0656	237 MiB (0.37)
Q4_K_S	0.995228	75/100	0.928064	0.0709	228 MiB (0.36)

4 Validation Methodology

Because PLM ports fail silently, we validate bottom-up rather than by a single end-to-end check:

M4: token IDs $\rightarrow$ M5: layer-0 Q/K L2 norms $\rightarrow$ M6: full 30-layer per-residue cosine.

The pass thresholds are: exact token-ID match against the official tokenizer; relative L2 error $\leq 5\%$ for Q and K after layer 0 (post-scale, post-RoPE); per-residue mean cosine > 0.999 and minimum cosine > 0.99 against a PyTorch reference (with `<cls>/<eos>`

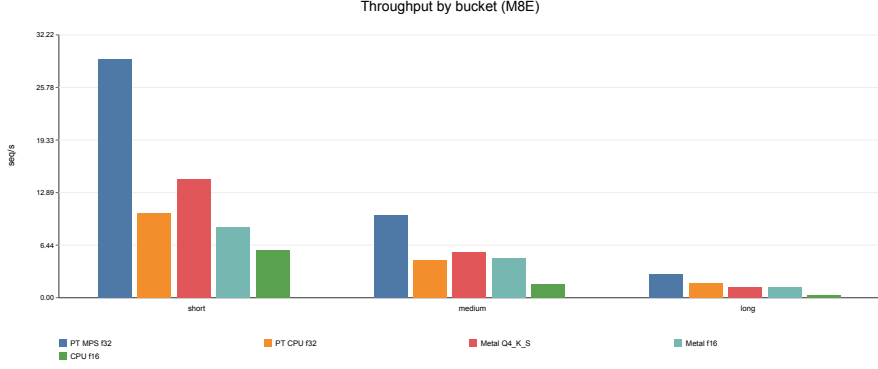


Figure 1: Single-sequence throughput (seq/s) by length bucket for `esmc.cpp` Metal configurations on M4 Max. Higher is better.

Table 4: Single-sequence throughput (seq/s) on M4 Max Metal (300M, 1000 iterations).

Precision	short (47t)	medium (235t)	long (850t)
F16	91.7	36.9	5.8
Q8_0	82.6	32.8	5.4
Q4_K_M	84.0	35.6	5.3
Q4_K_S	86.2	31.3	4.7

Q8_0 is a safe default: it passes the strict per-sequence threshold on all 100 sequences at $0.53\times$ the F16 size. Both 4-bit schemes pass the aggregate 0.995 target, but not on every individual sequence, and failures concentrate in short sequences (for Q4_K_M, 9 of 9 failures are in the small bucket). This is consistent with quantization error being a roughly fixed absolute perturbation that matters more when there are fewer residues to average over. Q4_K_M dominates Q4_K_S (91 versus 75 passes) and is the recommended 4-bit choice for this architecture.

5.2 Throughput

Table 4 summarizes single-sequence throughput on M4 Max Metal across precisions. `esmc.cpp` F16 Metal is the fastest configuration on this hardware, reaching 91.7 seq/s on short sequences

Flash attention (M2). The dense $QK^T \rightarrow \text{softmax} \rightarrow KQV$ sequence was replaced with `ggml_flash_attn_ext`, reducing compute time by up to 24% on the longest bucket (908 ms $\rightarrow$ 689 ms at 2002 tokens). A dense fallback remains selectable via a runtime flag.

GPU scheduler (M3). Introducing `ggml_backend_sched` keeps all intermediate tensors GPU-resident across calls, reducing first-allocation latency by 99% and steady-state compute time by 10–19%.

Precision cleanup (M2.2). Forced `GGML_PREC_F32` on the dense attention matmuls was removed: on Metal the setting was a no-op (the backend always accumulates in F32), and removing it improves forward-compatibility for CPU backends that may use F16 accumulation.

5.4 Batching

To optimize for throughput-bound workloads such as FASTA batch embedding and ProteinGym sweeps, we extended `esmc.cpp` with a batched inference path (`esmc_embed_batch`) that accepts flat 1D token input $[\text{max_len} \times n_{\text{seq}}]$, a shared position tensor, and an F16 attention mask $[\text{max_len}, \text{max_len}, 1, n_{\text{seq}}]$. The graph is built with 3D tensors $[n_{\text{embd}}, \text{max_len}, n_{\text{seq}}]$ throughout; a 4D reshape is used only inside the attention operation. The backend scheduler caches the graph per distinct $(\text{max_len}, n_{\text{seq}})$ pair so rebuild overhead is paid only on the first call for each configuration.

A key design decision is the layout of the attention output. For the flash path, `ggml_cont_2d` followed by `ggml_reshape_3d` is correct; for the dense fallback, only the permute-back pattern (`ggml_permute(0, 2, 1, 3)  $\rightarrow$  ggml_cont`) produces the right ordering — `cont_2d` would map elements to wrong positions. Both paths reuse the same F16 mask tensor.

On M4 Max Metal with a 28-token sequence, throughput scales nearly linearly up to batch 4 ($2.8\times$ for $4\times$ the work) and reaches $3.6\times$ single-sequence throughput at batch 16 (345 seq/s). Beyond batch 16, GPU memory pressure introduces high variance (batch 32 yields 122–325 seq/s). Batched and per-sequence embeddings agree to within F16 noise (max difference 4.2×10^{-4}).

5.5 Memory Footprint

Figure 2 shows long-bucket peak resident set size (RSS) across all 36 configurations measured with `/usr/bin/time -l` in fresh processes. Every configuration fits comfortably within the 36 GB budget; the worst case (CPU F32, long) peaks at only 2609 MiB, or 7.1% of the machine. On M4 Max, peak RSS is dominated by

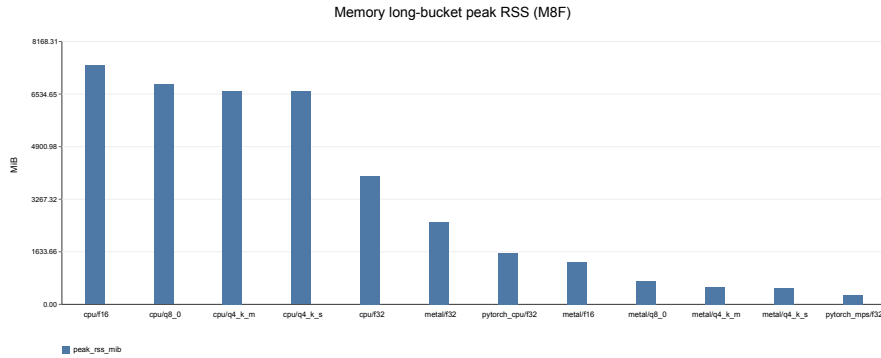
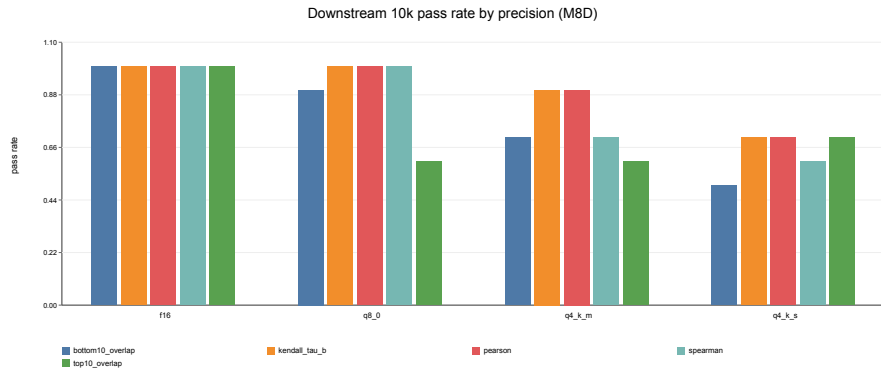


Figure 2: Long-bucket peak resident set size (MiB) by backend/precision. RSS scales with model file size; CPU and Metal are nearly identical per precision. Lower is better.



6 Discussion and Limitations

Scope. The quantitative study covers only the 300M model on a single M4 Max (36 GB) host. The 600M and 6B variants are not yet ported or benchmarked; in particular, 6B has $d_{\text{model}} = 2560$ (a multiple of 256), so we expect k -quant to behave qualitatively differently and the memory story to be the binding constraint rather than throughput.

Statistical significance. Each correctness, throughput, and memory number is a single run on a single machine; we report run-to-run throughput stability of within $\pm 2\%$ from a repeated F16 measurement but do not provide formal error bars or multi-host variance. Headline claims should be read as point estimates on this hardware.

k -quant limitation. At $d_{\text{model}} = 960$, only one of seven weight matrices per layer uses true k -quant blocks; the rest use legacy fallbacks. This both caps the achievable compression and explains the short-sequence quality concentration. Claims about 4-bit quality should be qualified by the 75–91% per-sequence pass rate or restricted to sequences above a minimum length.

Throughput regime. `esmc.cpp` is a portability and memory-efficiency path, not a universal throughput win: on M4 Max, F16 Metal reaches 36.9 seq/s on medium sequences, and the CPU path is 10–40 $\times$ slower. Flash attention, GPU-resident graph scheduling, and multi-sequence batching (Sections 5.3 and 5.4) have been implemented and measured, addressing the $O(n^2)$ attention bottleneck and per-forward overhead. The remaining constraints are the single-hardware scope (36 GB M4 Max only) and the 300M-only model restriction; batch 32 throughput collapses under GPU memory pressure, so the sweet spot is batch 16.

Downstream proxy. The downstream task measures whether quantization *preserves* the PyTorch embedding behavior, using a cosine-based proxy rather than ProteinGym’s masked-marginal likelihood. It supports preservation claims for F16 and Q8_0 rank metrics but does not establish absolute biological prediction quality, and does not support blanket 4-bit preservation claims under the 0.01 Spearman tolerance.

7 Conclusion

We ported ESM-C 300M to `esmc.cpp`, a dependency-minimal C/C++ runtime on the ggml/llama.cpp stack, with a GGUF schema and converter for the native checkpoint layout and a staged validation pipeline designed to catch silent numerical failures. The F16 runtime reproduces PyTorch embeddings to a mean cosine of 0.99999, 8-bit quantization preserves 0.99971 at half the model size, and quantized models occupy as little as 228 MiB on disk with a 520 MiB peak resident footprint. A completed performance roadmap – flash attention, GPU-resident graph scheduling, and multi-sequence batching (Sections 5.3 and 5.4) – addresses the long-sequence bottleneck and achieves 3.6 $\times$ single-sequence throughput at batch 16 on M4 Max hardware. The bug case studies argue that alphabet-covering test sequences and bottom-up numeric probes are necessary, not optional, when porting protein language models. Extending the runtime to 600M and 6B, multi-hardware benchmarking, and optimizing batched throughput beyond batch 16 remain as future work.

Acknowledgments and Disclosure of Funding

The DataEngUtils macOS application provided the CSV, JSON,

- [3] Su, J., Lu, Y., Pan, S., Wen, B. & Liu, Y. (2021) RoFormer: Enhanced transformer with rotary position embedding. *arXiv:2104.09864*.
- [4] Shazeer, N. (2020) GLU variants improve transformer. *arXiv:2002.05202*.
- [5] Ba, J.L., Kiros, J.R. & Hinton, G.E. (2016) Layer normalization. *arXiv:1607.06450*.
- [6] Gerganov, G. and the llama.cpp contributors (2023–2025) llama.cpp and the ggml tensor library. <https://github.com/ggml-org/llama.cpp>
- [7] llama.cpp contributors (2023–2025) GGUF: a single-file format for ggml models. <https://github.com/ggml-org/ggml/blob/master/docs/gguf.md>
- [8] Notin, P., Kollasch, A.W., Ritter, D., van Niekerk, L., Paul, S., Spinner, H., et al. (2023) ProteinGym: Large-scale benchmarks for protein fitness prediction and design. In *Advances in Neural Information Processing Systems* 36.
- [9] The UniProt Consortium (2023) UniProt: the universal protein knowledgebase in 2023. *Nucleic Acids Research* **51**(D1):D523–D531.
- [10] Tsuboyama, K., Dauparas, J., Chen, J., Laine, E., Mohseni Behbahani, Y., Weinstein, J.J., et al. (2023) Mega-scale experimental analysis of protein folding stability in biology and design. *Nature* **620**(7973):434–444.

A Reproduction and Tensor Naming

Code and data availability. The complete runtime, GGUF converter, quantizer, and benchmark harnesses are open source at <https://github.com/AnanyaP-WDW/esmc.cpp>. The pre-converted GGUF model files (F32/F16/Q8_0/Q4_K_M/Q4_K_S), together with a model card and the benchmark summary, are published at <https://huggingface.co/AnanyaPathak/esmc-300m-gguf>.

End-to-end reproduction (300M).

```
# Environment + weights (torch is needed for the PyTorch reference/baselines)
python3 -m venv .venv
.venv/bin/pip install -r tools/requirements.txt
.venv/bin/pip install -e ./ggml/gguf-py
.venv/bin/pip install torch
hf download EvolutionaryScale/esmc-300m-2024-12 --local-dir ./esmc-300m

# Build C++ (esmc-embed, esmc-bench, esmc-quantize)
cmake -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build -j8

# Convert (F16 + F32) + verify
.venv/bin/python tools/convert_esmc_to_gguf.py ./esmc-300m \
    ./models/esmc-300m-f16.gguf --dtype f16
.venv/bin/python tools/convert_esmc_to_gguf.py ./esmc-300m \
    ./models/esmc-30
```

```

--output tests/reference_embeddings.npz

# Benchmarks
.venv/bin/python benchmarks/correctness.py
.venv/bin/python benchmarks/throughput.py --config benchmarks/config_throughput_300m.json
.venv/bin/python benchmarks/memory.py      --config benchmarks/config_memory_300m.json
.venv/bin/python benchmarks/fetch_proteingym_subset.py --download \
    --selection-mode multi-assay --max-assays 10 --max-rows 1000 --min-rows 1000 \
    --max-length 512 --output-dir benchmarks/proteingym_subset_10k \
    --manifest benchmarks/datasets/proteingym_subset_10k_manifest.json
.venv/bin/python benchmarks/downstream.py --config benchmarks/config_downstream_300m_10k.json

```

Canonical GGUF tensor names. Global tensors are token_embd.weight, output_norm.weight, and output.weight; per-layer tensors follow the blk.{i}.* pattern (attn_q/k/v/output.weight, attn_norm.weight/.bias, attn_q_norm.weight, attn_k_norm.weight, ffn_norm.weight, ffn_gate/up/down.weight), for a total of 363 tensors at 300M.

Conference For AI Scientists 2026 - AI Involvement Checklist

Research Stage Assessment

1. **Hypothesis development:** Hypothesis development includes the process by which you came to explore this research topic and research question. This can involve the background research performed by either researchers or by AI. This can also involve whether the idea was proposed by researchers or by AI.

Answer: **[B]**

Iteration Effort: **[Medium]**

Explanation: The high-level idea (port an encoder-only protein language model to the ggml/llama.cpp stack and quantify the fidelity/efficiency trade-off) was framed jointly: humans set the goal and the consumer-hardware framing, while the AI system surveyed the ESM-C architecture, identified the ESM2 vs. ESM-C deltas that matter for correctness, and proposed the staged-validation and benchmark structure. Iteration was moderate, spanning several rounds of refining scope (300M-only) and benchmark choices.

2. **Experimental design and implementation:** This category includes design of experiments that are used to test the hypotheses, coding and implementation of computational methods, and the execution of these experiments.

Answer: **[C]**

Iteration Effort: **[High]**

Explanation: The AI system authored the large majority of the converter, the C++ ggml forward graph, the quantizer, and the benchmark harnesses, with humans providing direction, running commands on the target hardware, and approving milestone gates. Implementation required extensive trial-and-error (well over 100 substantive attempts), particularly around RoPE style, query scaling order, the fused-projection split, and the k -quant fallback policy.

3. **Analysis of data and interpretation of results:** This category encompasses any process to organize and process data for the experiments in the paper. It also includes interpretations of the results of the study.

Answer: **[C]**

Iteration Effort: **[Medium]**

Explanation: The AI system computed the cosine/L2 metrics, aggregated the correctness/throughput/memory/downstream tables, and drafted the interpretations (e.g., the short-sequence quantization concentration and the CPU-vs-Metal throughput split). Humans verified the numbers against raw artifacts and sanity-checked the conclusions.

4. **Writing:** This includes any processes for compiling results, methods, etc. into the final paper form. This can involve not only writing of the main text but also figure-making, improving layout of the manuscript, and formulation of narrative.

Answer: **[C]**

Iteration Effort: **[Medium]**

Explanation: The AI system drafted the full manuscript, tables, and figure captions from the project's lab notebook and result files, and converted the benchmark plots to the embedded figures. Humans guided the structure, enforced the template/format constraints, and reviewed

specialized scientific model was used beyond the ESM-C protein language model that is the subject of the port.

6. **Observed AI Limitations:** What specific limitations and failure modes have you observed when using AI as a partner or lead author?

Description: The dominant failure mode was *silent numerical error*: the agent produced code that ran and yielded plausible embeddings while being subtly wrong (e.g., a transposed vocabulary entry that only affected sequences containing certain amino acids, and an incorrectly ordered query scaling relative to RoPE). These were caught only by the staged, alphabet-covering validation harness, not by the agent’s own judgement. The agent also tended toward optimistic claims and required human gating to scope conclusions honestly (e.g., not over-claiming 4-bit quality or single-host throughput), and it occasionally needed correction on platform-specific edge cases such as the k -quant block-size constraint at $d_{\text{model}} = 960$.

Conference For AI Scientists 2026 - Reproducibility and Responsibility Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer: [Yes]

Justification: The abstract and Section 1 state the contributions (port, GGUF schema/converter, staged validation, 300M empirical study) and explicitly scope the study to 300M on a single M4 Max host, matching the results in Section 5 and the limitations in Section 6.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 6 discusses scope (300M only), absence of error bars, the k -quant structural limitation, the non-competitive CPU/long-sequence regime, and the downstream proxy caveat.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [NA]

Justification: The paper is a systems and empirical study; it contains no theorems or formal proofs.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 3, Section 4, Section 5, and Appendix A give the architecture, conversion, quantization mix, validation thresholds, datasets, hardware, and exact reproduction commands.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The runtime, converter, quantizer, and benchmark harnesses are released with the project, and the benchmark sequences are derived from public Swiss-Prot and ProteinGym data with provenance manifests; the model weights are publicly available ESM-C 300M checkpoints. Code: <https://github.com/AnanyaP-WDW/esmc.cpp>; pre-converted GGUF models and model card:

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: Results are single runs on a single host; we report run-to-run throughput stability ($\pm 2\%$) but do not provide formal error bars, and we flag this explicitly in Section 6.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: All experiments run on an Apple M4 Max with 36 GB unified memory (macOS 26.5); peak memory per configuration is reported in Section 5, and indicative runtimes (e.g., the downstream run) are noted.

9. **Research Ethics**

Question: Does the research conducted in the paper conform with the NeurIPS Code of Ethics, except where CAISc’s AI authorship policies apply?

Answer: [Yes]

Justification: The work uses public protein sequence data and open model weights, involves no human subjects or personal data, and conforms to the Code of Ethics except for the AI-authorship aspects governed by CAISc policy and documented in the AI Involvement Checklist.

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Lowering the deployment cost of protein embeddings broadens access for research and education (positive); as with any protein-modeling tool there is a dual-use consideration, but this work only makes an existing public model cheaper to run rather than increasing its capability, so the marginal risk is limited.